

아니메컵 1 쿨 풀이

Official Editorial

by

아니메컵 제작위원회

문제	의도한 난이도	출제자
A :chino_shock:	Easy	sait2000
B 치노의 라떼 아트 (Easy)	Easy	bubbler
C 윤영진에게 설정 짜기는 어려워	Easy	pani
D 효율적인 애니메이션 감상	Medium	dhyang24
E 러키☆한별	Medium	dhyang24
F 틀리는 건 싫으니까 쉬운 문제에 올인하려고 합니다	Medium	index
G 시간은 다시 움직인다	Medium	dhyang24

문제	의도한 난이도	출제자
H 복슬복슬 여우꼬리	Hard	martinok1103
I 세상에서 가장 달달한 디저트 만들기	Hard	pani
J 치노의 라떼 아트 (Hard)	Hard	bubbler
K 코코아.... 이거 아니라고	Hard	sait2000
L UMR의 테트리스 플레이 분석하기	Hard	bubbler
M 한별이의 퍼펙트 수열과 퀴리 교실	Challenging	index

1A. :chino_shock:

string

출제진 의도 - **Easy**

- 제출 714번, 정답 634명 (정답률 88.796%)
- 처음 푼 사람: **riroan**, 0분
- 출제자: **sait2000**

1A. :chino_shock:

- 이모지의 전체 길이, 콜론의 개수, 언더바의 개수를 알면 풀 수 있습니다.
- 이모지의 전체 길이는 문자열의 길이를 구하는 내장 함수 등을 사용하면 구할 수 있습니다.
- 콜론의 개수는 문제 조건에 의해서 언제나 2입니다.
- 언더바의 개수는 이모지의 각 문자에 대해 보면서 언더바인지 확인하면 구할 수 있습니다.

1B. 치노의 라떼 아트 (Easy)

implementation

출제진 의도 – Easy

- 제출 364번, 정답 75명 (정답률 20.604%)
- 처음 푼 사람: **xiaowuc1**, 6분
- 출제자: bubbler

1B. 치노의 라떼 아트 (Easy)

- 가장 간단하게 생각해서, 하트가 들어있는 모든 가능한 입력을 만들어 놓고 각각의 입력과 비교해보는 풀이를 생각해 봅시다.
- 이때 예상되는 시간 복잡도는

$$\begin{aligned} & (R \text{의 개수}) \times (C \text{의 개수}) \times (\text{하트의 왼쪽 위 좌표의 개수}) \times \\ & (N \text{의 개수}) \times (M \text{의 개수}) \times (\text{방향의 개수}) \times (\text{글자 비교 횟수}) \\ & = O(R^3 C^3 \min(R, C)^2) \end{aligned}$$

처럼 보이지만, 실제로 조건을 만족하는 입력의 개수는 약 4만 3천 개, 이를 모두 문자열로 저장했을 때 글자 수는 약 279만 개밖에 되지 않으며, 한 번 만들어 놓고 일일이 비교하더라도 대부분 크기 비교와 처음 몇 글자에서 걸러지기 때문에 널널하게 통과됩니다.

1B. 치노의 라떼 아트 (Easy)

- 앞의 풀이 외에도 다양한 풀이가 가능한 문제입니다.
- 출제자의 풀이는 다음과 같습니다.
 - 크림이 있는 모든 칸의 x 좌표의 최댓값 $x_{\max}$ 와 최솟값 $x_{\min}$, 그리고 y 좌표의 최댓값 $y_{\max}$ 와 최솟값 $y_{\min}$ 을 각각 구합니다.
 - 이 범위가 정사각형이면 ($x_{\max} - x_{\min} = y_{\max} - y_{\min} = N - 1$), 이 정사각형 내에서 크림이 없는 칸의 개수를 구합니다.
 - 이 개수가 제곱수 M^2 이면, 주어진 모양이 $N \times N$ 정사각형의 네 구석 중 한쪽에서 $M \times M$ 정사각형을 제거한 것과 일치하는지 확인합니다.

1B. 치노의 라떼 아트 (Easy)

- 검수진의 풀이 중 몇 가지를 요약하면 다음과 같습니다.
- 풀이 1
 - 입력 전체에서 #의 개수가 $N^2 - M^2$ 일 때, 한 변의 길이가 N 인 정사각형의 모든 가능한 위치에 대해서 다음의 조건을 테스트하여 한 번이라도 통과하면 하트로 판정
 - ▶ 현재 정사각형에 포함되는 #의 개수가 $N^2 - M^2$
 - ▶ 현재 정사각형의 네 구석 중 한 곳의 $M \times M$ 영역에 포함되는 #의 개수가 0
 - 사각 영역의 #의 개수를 셀 때 2차원 누적합을 사용할 수 있습니다.

1B. 치노의 라떼 아트 (Easy)

— 풀이 2

- #. 모양이 한 번만 등장함을 확인 후 정사각형 조건을 판별
- 위 패턴이 있는 곳의 위치와 #의 최대, 최소 x, y 좌표를 조합하면 입력이 하트라고 가정했을 때 그 하트를 구성하는 두 정사각형의 위치와 크기를 모두 알 수 있습니다.
- 입력이 실제로 그 하트가 맞는지 한 번 더 검사한 후 결과를 출력하면 됩니다.

1C. 운영진에게 설정 짜기는 어려워

ad_hoc

출제진 의도 – Easy

- 제출 182번, 정답 66명 (정답률 36.264%)
- 처음 푼 사람: **xiaowuc1**, 11분
- 출제자: pani

1C. 윤영진에게 설정 짜기는 어려워

- 참고 대상인 캐릭터가 한 명 밖에 없는 경우를 생각해봅시다.
 - 첫 번째 캐릭터의 첫 번째 속성의 값을 알아내고, 신규 캐릭터의 첫 번째 속성의 값을 이와 다르게 설정합니다.
 - 나머지 속성을 어떻게 설정하더라도 첫 번째 캐릭터와 설정이 겹치지 않는 캐릭터를 만들 수 있습니다.

1C. 윤영진에게 설정 짜기는 어려워

- 참고 대상인 캐릭터가 두 명인 경우에는 어떻게 될까요?
 - 위와 같이 첫 번째 캐릭터의 첫 번째 속성의 값을 알아낸 후, 두 번째 캐릭터의 두 번째 속성 값을 알아냅니다.
 - 신규 캐릭터의 첫 번째 속성 값은 첫 번째 캐릭터의 첫 번째 속성의 값과 다르게 설정합니다.
 - 신규 캐릭터의 두 번째 속성 값은 두 번째 캐릭터의 두 번째 속성의 값과 다르게 설정합니다.
 - 나머지 속성을 어떻게 설정하더라도 앞선 두 캐릭터와 설정이 겹치지 않는 캐릭터를 만들 수 있습니다.

1C. 윤영진에게 설정 짜기는 어려워

- 이제 문제 조건과 같이 참고 대상인 캐릭터가 M 명이 있는 경우를 생각해봅시다.
- b_{ij} 를 i 번째 캐릭터의 j 번째 속성의 값이라고 한다면, 각 캐릭터의 속성의 값을 다음과 같이 나열할 수 있습니다.
 - $b_{11}b_{12}b_{13}b_{14}b_{15}b_{16}b_{17}b_{18}\cdots$
 - $b_{21}b_{22}b_{23}b_{24}b_{25}b_{26}b_{27}b_{28}\cdots$
 - $b_{31}b_{32}b_{33}b_{34}b_{35}b_{36}b_{37}b_{38}\cdots$
 - $b_{41}b_{42}b_{43}b_{44}b_{45}b_{46}b_{47}b_{48}\cdots$
 - $b_{51}b_{52}b_{53}b_{54}b_{55}b_{56}b_{57}b_{58}\cdots$
 - $b_{61}b_{62}b_{63}b_{64}b_{65}b_{66}b_{67}b_{68}\cdots$

1C. 윤영진에게 설정 짜기는 어려워

- 참고 대상인 캐릭터가 두 명 일때와 같은 방법으로, 각 캐릭터에서 서로 겹치지 않는 속성 하나씩을 알아냅니다.
- 한 가지 경우로, M 번의 질문을 통해서 속성 $b_{11}, b_{22}, \dots, b_{MM}$ 의 값을 알아낼 수 있습니다.
 - 이 작업을 위해서는 $M \leq Q, M \leq N$ 이어야 하며, 문제 조건에 따라 이를 만족합니다.
- i 가 M 이하라면 p_i 는 위에서 구한 b_{ii} 와 다른 임의의 값으로, i 가 M 초과라면 범위 내의 임의의 값으로 설정하는 것으로 조건에 맞는 새로운 캐릭터를 만들 수 있습니다.

1D. 효율적인 애니메이션 감상

greedy, parametricsearch

출제진 의도 – **Medium**

- 제출 328번, 정답 39명 (정답률 11.890%)
- 처음 푼 사람: **leedongbin**, 11분
- 출제자: dhyang24

1D. 효율적인 애니메이션 감상

- 몇 개의 애니메이션을 볼 수 있을지 모르는 상태로 애니메이션 감상을 최적화 하는 것은 어렵습니다.
- 따라서, 애니메이션의 개수를 고정하고 그만큼의 애니메이션을 시간 안에 볼 수 있는지 결정하는 문제로 변환하여, 애니메이션의 개수에 대하여 매개 변수 탐색을 할 수 있습니다.
- 우선, A 개의 애니메이션을 볼 수 있을 때 길이가 가장 짧은 A 개를 선택하는 것이 가장 빠르다는 것은 자명합니다.
- 또한, 애니메이션을 보는 데에 걸리는 총 시간은 애니메이션을 동시에 볼 그룹으로 나누었을 때, 각 그룹의 가장 긴 애니메이션을 보는 데에 걸리는 시간의 합과 같습니다.

1D. 효율적인 애니메이션 감상

- 가장 긴 애니메이션부터 K 개씩 그룹으로 묶는 것이 최적이라는 것을 증명할 수 있습니다.
- K 개씩 그룹으로 묶었을 때 시간 안에 애니메이션을 볼 수 있는지 판별하는 것은 $O(N)$ 시간 안에 할 수 있습니다. 매개 변수 탐색에 이분 탐색을 사용하면 총 시간복잡도는 $O(N \log N)$ 입니다.
- 매개 변수 탐색을 이용하지 않고 DP를 이용하는 풀이도 존재합니다.

1E. 러키☆한별

bfs

출제진 의도 – **Medium**

- 제출 71 번, 정답 27명 (정답률 38.028%)
- 처음 푼 사람: **xiaowuc1**, 22분
- 출제자: dhyang24

1E. 러키☆한별

- 한별이를 제외한 각 사람들이 취할 수 있는 최선 전략은 한별이가 가고자 하는 출구에 미리 가서 한별이를 기다리는 것입니다.
- 따라서, 어떤 사람이 한별이보다 먼저 한별이가 탈출하고자 하는 출구에 도착할 수 있다면, 항상 선물을 줄 수 있습니다.
- 사람과 출구의 수가 많지 않으므로 각 사람에 대하여 출구까지의 거리를 구하고, 각 출구에 대하여 한별이보다 그 출구에 빨리 도착할 수 있는 사람의 수를 구하면, 가장 선물을 많이 받을 수 있는 출구를 구할 수 있습니다.
- 각 사람마다 BFS를 실행할 경우 BFS는 $O(NM)$ 시간이 걸리고, 사람의 수가 적당히 작으므로 총 시간복잡도는 $O(NM)$ 입니다.
- 상수가 넉넉한 편은 아니므로 지나치게 비효율적인 구현은 시간제한 안에 들어오지 않을 수도 있습니다.

1F. 틀리는 건 싫으니까 쉬운 문제에 올인하려고 합니다

greedy, data structures, priority queue

출제진 의도 – **Medium**

- 제출 77번, 정답 24명 (정답률 31.169%)
- 처음 푼 사람: **kimjihoon**, 66분
- 출제자: index

1F. 틀리는 건 싫으니까 쉬운 문제에 올인하려고 합니다

- 우선 모든 문제의 데이터와 에디토리얼이 없는 상태라고 가정해봅시다.
- 이 경우 현재 아이디어 난이도가 한별이의 능력 이하인 문제들만 풀 수 있습니다.
- 풀 수 있는 문제들 가운데서는 구현 난이도가 낮은 문제부터 푸는 것이 항상 이득입니다.
 - 구현 난이도가 높은 문제를 먼저 풀게 되면 받는 **틀렸습니다**의 개수가 그대로거나 더 늘어나지만 한별이의 실력은 구현 난이도가 낮은 문제를 풀 때와 같은 양만큼 늘어나기 때문입니다.
- 그러나 이 과정을 단순히 구현하면 **시간 초과**가 되므로 '우선순위 큐' 등의 자료 구조를 이용해 풀 수 있는 문제 중 구현 난이도가 가장 낮은 것을 빠르게 뽑아야 합니다.

1F. 틀리는 건 싫으니까 쉬운 문제에 올인하려고 합니다

- 이제 각 문제의 데이터와 에디토리얼이 있는 경우 어떻게 할지 알아보시다.
 - 데이터가 있는 경우 단순히 구현 난이도를 0으로 만들면 됩니다.
 - ▶ 구현 난이도가 한별이의 구현 능력 이하라면 **틀렸습니다**를 받지 않기 때문입니다.
 - 에디토리얼이 있는 경우는 아이디어 난이도를 에디토리얼을 이해하는 난이도인 $\lceil \text{원래의 아이디어 난이도} / 2 \rceil$ 로 바꾼 다음, 구현 난이도를 바뀌는 난이도인 $\lceil \text{원래의 구현 난이도} / 2 \rceil$ 로 바꾸면 됩니다.
 - ▶ 바뀌는 아이디어 난이도가 더 낮기는 하지만 에디토리얼을 이해하지 못하면 의미가 없으므로 이해하는 난이도가 기준점이 됩니다.

1G. 시간은 다시 움직인다

dp

출제진 의도 – **Medium**

- 제출 94번, 정답 14명 (정답률 14.894%)
- 처음 푼 사람: **yclock**, 30분
- 출제자: dhyang24

1G. 시간은 다시 움직인다

- 능력을 사용할 때마다 능력 지속 시간이 길어지므로, 능력 사용 횟수는 $\sqrt{M}$ 에 비례합니다.
여기서 M 은 가장 늦은 고비가 오는 시각 t_N 을 의미합니다.
- 따라서, 매 시각과 능력을 사용하는 횟수를 매개 변수로 하여 DP를 하면 $O(M\sqrt{M})$ 으로, 시간 안에 작동하는 솔루션을 만들 수 있습니다.

1G. 시간은 다시 움직인다

- $DP[a][b]$ 를 능력을 a 번 사용하고, b 시각에 다시 능력을 사용할 수 있으며, b 이전에 오는 고비를 모두 수월하게 넘기면서 b 라는 시각에 도달할 수 있으면 1, 아니면 0으로 정의합니다.
- 모든 고비를 수월하게 넘기려면, 능력이 끝나고 다시 능력을 사용할 수 있도록 기다리는 시간에 고비가 찾아오면 안 됩니다.
 - b 시각에 지속시간이 $a + 1$ 이 된 능력을 사용하려면, $b + a + 1$ 이상 $b + 2a + 2$ 미만 시각에 찾아오는 고비가 없어야 합니다.
 - 이 조건을 빠르게 확인하기 위한 방법으로, 어떤 시각 이전까지에 찾아오는 고비의 개수를 부분합을 통해서 구해놓는 방법이 있습니다.
- $DP[a][b] = 1$ 일 때, b 에 찾아오는 고비가 없는 경우 $DP[a][b + 1] = 1$ 이고, 해당 시각에 능력을 사용할 수 있다면 $DP[a + 1][b + 2a + 2] = 1$ 입니다. 따라서 b 가 커지는 방향으로 bottom-up 방식으로 계산할 수 있습니다.

1H. 복슬복슬 여우꼬리

prefix sum, binary search, case work

출제진 의도 – **Hard**

- 제출 56번, 정답 7명 (정답률 12.500%)
- 처음 푼 사람: **gs18115**, 98분
- 출제자: martinok1103

1H. 복슬복슬 여우꼬리

- 여우 신이 강림해 있는 0시부터 M 시까지 사이에서 각 푸석푸석한 시간대별로 몇 번의 마법을 사용할 수 있는지에 대한 배열을 생성합니다.
 - 구간별로 푸석푸석한 시간대의 길이를 저장한 배열을 D 라고 했을 때, $D_i = X_i - Y_{i-1}$ 이 됩니다.
 - 이때 $D_0 = X_0, D_N = M - Y_{N-1}$ 로 둡니다.
- 각 시간대 D_i 마다 몇 번의 마법을 사용할 수 있는지 계산합니다. 시간대별 사용 횟수 배열을 $DCnt$ 라고 했을 때, $DCnt_i = \left\lceil \frac{D_i}{T} \right\rceil$ 가 됩니다.

1H. 복슬복슬 여우꼬리

- 푸석푸석한 시간대에 계속 마법을 사용한다고 가정하여 순차적으로 마법 사용에 대한 번호를 부여했을 때 $DCnt_i$ 시간대별로 시작 번호는 S_i , 종료 번호는 E_i 로 둡니다.
 - $S_0 = 1$ 로 두고, $E_0 = DCnt_0$ 으로 둡니다.
 - 이후의 값들에 대해서 $S_i = E_{i-1} + 1$ 로, $E_i = E_{i-1} + DCnt_i$ 로 둘 수 있습니다.
 - 예를 들어, $DCnt_0 = 4, DCnt_1 = 3$ 라고 가정하면, $S_0 = 1, E_0 = 4, S_1 = 5, E_1 = 7$ 이 됩니다.
- 이때 $E_n \leq K$ 인 경우에 대해서는 모든 시간대에 대해서 마법을 사용하면 되므로, 답은 M 이 됩니다.

1H. 복슬복슬 여우꼬리

- $E_n > K$ 인 경우에 대해서는 1 번부터 E_N 번 사이의 구간에서 연속된 K 개의 구간에 마법을 사용하였을 때의 복슬복슬 시간의 최댓값을 구해줍니다.
- 마법이 사용될 구간의 번호가 i 번부터 $i + K - 1$ 번 구간이라고 할 때, i 와 $i + K - 1$ 이 몇 번째 구간에 속하는지 S 와 E 배열에서 이분탐색을 통해 찾아 각각의 구간 번호를 구합니다.
 - 시작 지점이 복슬복슬 시간과 붙어있는 경우, 직전 복슬복슬 시간을 마법 적용 시간에 포함하여 계산합니다.
 - 종료 지점이 복슬복슬 시간과 붙어있는 경우, 직후 복슬복슬 시간을 마법 적용 시간에 포함하여 계산합니다.

11. 세상에서 가장 달달한 디저트 만들기

math, exponentiation_by_squaring

출제진 의도 – **Hard**

- 제출 31 번, 정답 18명 (정답률 58.065%)
- 처음 푼 사람: **gs18115**, 37분
- 출제자: pani

11. 세상에서 가장 달달한 디저트 만들기

- 초기 정육면체의 한 변의 길이가 N^M 이므로, 최종 상태에서의 가장 작은 정육면체의 한 변의 길이는 1 입니다.
- 과정을 M 번 반복한 후의 상태에서 가장 작은 정육면체의 개수를 V_M , 겉면에 보이는 가장 작은 정사각형의 개수를 S_M 이라고 합시다.
- 최종 상태에서의 부피와 겉넓이를 구하는 것은 V_M 과 S_M 을 구하는 것과 같습니다.

11. 세상에서 가장 달달한 디저트 만들기

- 먼저 부피를 계산해봅시다.
- 과정을 한 번 진행하게 되면, 이전 과정에서의 가장 작은 정육면체마다 꼭짓점에 각각 1 개씩 총 8 개, 모서리에 각각 $N - 2$ 개씩 총 $12 \times (N - 2)$ 개의 정육면체가 생기게 됩니다.
- 이를 정리하면 $V_M = (12N - 16)V_{M-1}$ 이고, $V_0 = 1$ 이므로 $V_M = (12N - 16)^M$ 이 됩니다.

11. 세상에서 가장 달달한 디저트 만들기

- 이제 겹넓이를 계산해봅시다.
- 과정을 한 번 진행하게 되면, 이전 과정에서의 각 정사각형 면마다 $4N - 4$ 개의 작은 정사각형이 생깁니다.
- 또한, 이전 과정에서의 정육면체마다 $6 \times 4 \times (N - 2) \times V_{M-1}$ 만큼 작은 정사각형이 생깁니다.
- 이를 정리하면

$$S_M = (4N - 4)S_{M-1} + (24N - 48)V_{M-1} = (4N - 4)S_{M-1} + (24N - 48)(12N - 16)^{M-1}$$

이 됩니다.

11. 세상에서 가장 달달한 디저트 만들기

- $S_M = (4N - 4)S_{M-1} + (24N - 48)(12N - 16)^{M-1}$ 의 양변을 $(4N - 4)^M$ 으로 나누어 주면 $\frac{S_M}{(4N - 4)^M} = \frac{S_{M-1}}{(4N - 4)^{M-1}} + \left(\frac{24N - 48}{4N - 4}\right) \left(\frac{12N - 16}{4N - 4}\right)^{M-1}$ 이 됩니다.
- 등비수열의 합 공식을 적용하는 것으로 위 점화식으로부터 S_M 의 일반항을 계산할 수 있습니다.
- 일반항을 계산하는 과정에서 모듈러 역원을 사용해야 할 수 있으며, M 의 범위가 매우 크기 때문에 분할 정복을 이용한 거듭제곱을 통해 시간 내에 부피와 겉넓이를 계산할 수 있습니다.
- 이외에 일반항을 직접 구하지 않고 행렬 식을 세운 뒤, 행렬의 빠른 거듭제곱을 이용하여 문제를 해결하는 방법도 있습니다.

1J. 치노의 라떼 아트 (Hard)

geometry

출제진 의도 – Hard

- 제출 55번, 정답 17명 (정답률 30.909%)
- 처음 푼 사람: **yclock**, 20분
- 출제자: bubbler

1J. 치노의 라떼 아트 (Hard)

- 주어진 다각형을 선분 AB 로 나눴을 때 양쪽이 볼록다각형이라는 점으로부터, A 와 B 를 제외한 모든 꼭짓점의 내각이 180 도보다 작다는 것을 알 수 있고, 따라서 내각이 180 도보다 큰 꼭짓점은 유일하며 그 점이 B 임을 알 수 있습니다.
- 내각이 정확히 180 도인 꼭짓점은 없으므로, 선분 AB 로 나뉜 두 다각형이 서로 대칭이 되려면 양쪽의 꼭짓점의 개수가 서로 같아야 하고, 따라서 A 는 다각형 위의 꼭짓점들을 시계 방향으로 보았을 때 B 로부터 정확히 $N/2$ 번째 후에 위치한 점이어야 합니다.
- N 이 홀수이면 하트일 수 없습니다.

1J. 치노의 라떼 아트 (Hard)

- 이제 A 와 B 를 제외한 나머지 꼭짓점들이 AB 에 대해 대칭인 위치에 있는지를 판별하면 됩니다. 실수 오차 없이 M 과 N 이 AB 에 대해 대칭임을 보이려면, $\overline{AM}^2 = \overline{AN}^2$, $\overline{BM}^2 = \overline{BN}^2$, $\overline{MN}$ 과 $\overline{AB}$ 가 수직임 3가지 중에서 2가지 이상을 체크하면 됩니다.
- 선분 길이의 제곱은 피타고라스 정리를 쓰고, 수직 판별은 내적이 0인지 계산하면 되며, 모두 `int64` 범위 내에서 계산할 수 있습니다.

1J. 치노의 라떼 아트 (Hard)

- 여담: 치노의 라떼 아트 시리즈는 치노가 애니에서 만들었다고 전해지는 아래의 라떼 그림에서 아이디어를 얻었습니다.



1K. 코코아.... 이거 아니라고

ad_hoc

출제진 의도 – Hard

- 제출 37번, 정답 10명 (정답률 27.027%)
- 처음 푼 사람: **kdh9949**, 10분
- 출제자: sait2000

1K. 코코아.... 이거 아니라고

- 편의상 일반성을 잃지 않고 $P < Q$ 라 합시다. 부등호가 반대라면 책 순서를 반대로 생각하면 됩니다.
- 맨 처음 치노가 꽃아야 할 책 B_1 을 생각해봅시다. $Q - P = 2$ 라면 고민할 것이 없으므로 $Q - P > 2$ 라 합시다.
- 이때, 치노는 $P + 1$ 번째 책을 꽃아야 함을 증명할 수 있습니다.

1K. 코코아.... 이거 아니라고

- 만약, $B_1 \neq P + 1$ 이면 어떻게 될까요?
- 가능한 경우를 다음과 같이 나뉘볼 수 있습니다.
 - $P + 1 < B_1 < Q - 1$
 - $B_1 = Q - 1$

1K. 코코아.... 이거 아니라고

- $P + 1 < B_1 < Q - 1$ 일 경우, $A_3 = B_1 + 1, A_4 = B_1 - 1$ 이면 B_3 으로 가능한 값이 없습니다.
- $B_1 = Q - 1$ 일 경우, $A_3 = B_1 - 1$ 이면 $A_2 = Q = B_1 + 1$ 이므로 B_2 로 가능한 값이 없습니다.

1K. 코코아.... 이거 아니라고

- 치노가 $B_1 = P + 1$ 을 꽃으면, 코코아는 $A_2 = Q$ 를 꽃고, A_3 을 고르게 됩니다.
- 이때, 처음 꽃힌 두 책 $A_1 = P$ 와 $B_1 = P + 1$ 을 제외하고 생각하면, 나머지 책은 번호가 $P - 1$ 이하이거나, $P + 2$ 이상입니다.
- 즉, 번호를 크기 순서로 나열하면 홀수와 짝수가 번갈아 나타납니다.
- 따라서, 편의상 책 번호가 $P + 2$ 이상인 책들의 번호를 모두 각각 2씩 작게 만들면, 원래 문제 상황에서 N 이 1 작아진 형태로 환원이 가능함을 알 수 있습니다.
- 이를 반복하면, $N = 1$ 인 상황까지 문제가 환원되고, 그 뒤는 자명합니다.

1K. 코코아.... 이거 아니라고

- 실제로 인덱스를 매번 바꾸는 $\mathcal{O}(N^2)$ 풀이는 시간 초과가 발생합니다.
- Linked list나 union-find를 사용하면 $\mathcal{O}(N)$ 에 구현할 수 있습니다.
- Fenwick tree나 set을 사용하면 $\mathcal{O}(N \log N)$ 에도 할 수 있습니다.

1K. 코코아.... 이거 아니라고

TMI

- 이 문제의 제목은, 도서관에 다 같이 온 기념으로, 치노가 예전에 매우 재밌게 본 나머지 내용을 줄줄이 꿰고 있는 책을 다시 읽고 싶어서 열심히 찾았지만 못 찾고 있던 참에, 그 책이 무엇인지 알게 된 거 같다고 주장한 코코아가 치노에게 도스토옙스키의 『죄와 벌』을 들려줬을 때 치노가 말한(속으로 생각한?) 대사의 코믹스 정식 한국어판 번역입니다. [BOJ 2031](#)과는 다르게 이번 문제 제목은 정식 한국어판을 따랐습니다.
- 문제의 사진은 Sait2000이 산 책들입니다. 9 권은 아직 뜯지도 않았습니니다. 10 권 정발한 건 에디토리얼 쓰면서 알았습니다.
- 이 문제는 원래 인터랙티브로 나올 뻔 했습니다. 인터랙티브로 $N = 10^6$ 이 될 거 같지가 않아서 함수 구현으로 바꿨습니다.

1L. UMR의 테트리스 플레이 분석하기

implementation, prefix sum, bruteforcing

출제진 의도 – **Hard**

- 제출 5번, 정답 1명 (정답률 20.000%)
- 처음 푼 사람: **mjhmjh1104**, 214분
- 출제자: bubbler

1L. UMR의 테트리스 플레이 분석하기

- 편의상 테트로미노를 놓기 전의 필드를 *before*, 놓은 후의 필드를 *after*라고 하겠습니다.
- 그리고 각 필드의 맨 윗줄의 y 좌표를 0, 맨 아랫줄을 $H - 1$ 이라고 하고, *before* 또는 *after*의 i 번째부터 j 번째 가로줄 (양 끝 포함)을 각각 $\text{before}[i..j]$, $\text{after}[i..j]$ 라고 합시다.

1L. UMR의 테트리스 플레이 분석하기

- 일단 몇 줄이 지워졌는지 확인합니다.
- 이는 before와 after에서의 블록의 개수를 이용해 구할 수 있습니다. 전자를 x , 후자를 y 라고 하면 지워진 줄 수는 $c = \frac{x + 4 - y}{W}$ 입니다. c 는 0 이상 4 이하의 정수여야 하며, $\text{after}[0..c-1]$ 은 완전히 비어 있어야 합니다.

1L. UMR의 테트리스 플레이 분석하기

- 어떤 테트로미노를 놓는가에 따라 before에서 최대 연속 4줄의 상태가 바뀔 수 있고, 그 외의 영역은 $\text{before}[0..H-1]$ 와 $\text{after}[c..H-1]$ 가 동일해야 합니다.
- 이를 조금 더 엄밀하게 표현하면, 어떤 y 에 대해서 테트로미노를 놓고 줄을 지우는 동작에 의해 $\text{before}[y..y+3]$ 이 $\text{after}[y+c..y+3]$ 로 바뀌었을 때,
 $\text{before}[0..y-1] = \text{after}[c..y+c-1], \text{before}[y+c..H-1] = \text{after}[y+c..H-1]$ 여야 합니다.
- 어느 4줄이 바뀌었는가는 필드 전체의 세로 좌표에 대해 브루트포스를 시행하고, 그 외의 영역이 동일한지 판단은 필드 위아래 끝에서부터 시작하는 *prefix and* 를 계산해놓고 사용하면 제공 복잡도를 선형으로 줄일 수 있습니다.

1L. UMR의 테트리스 플레이 분석하기

- `before[y..y+3]` 내에서 어느 줄이 지워졌는지도 모든 가능한 경우를 고려해야 합니다.
- 각 경우에 대해 `before`와 `after`의 블록의 변화를 보고, 블록이 4개 추가되었다면 그 모양이 테트로미노가 맞는지, 맞는다면 어느 모양인지 판단합니다.
- 최소한의 하드코딩으로 테트로미노 모양 판별을 하려면, 4개의 블록의 좌표를 일종의 “대푯값”으로 변환하여 7가지 모양의 대푯값과 비교하는 방법이 있습니다. 출제자는 이 과정을 `canonicalize` 알고리즘이라고 부릅니다. 조금 더 자세한 설명은 다음 페이지를 참조하세요.
- 테트로미노를 공중에 놓을 수 없다는 규칙 판별도 잊지 마세요.

1L. UMR의 테트리스 플레이 분석하기

- canonicalize 알고리즘: 테트로미노의 각 블록의 x, y 좌표가 주어질 때, 다음의 과정을 순서대로 거쳐서 모양에 대한 정보만 남깁니다.
- 먼저 평행이동에 대한 정보를 제거합니다. 모든 블록의 x 좌표의 최솟값 $x_{\min}$ 과 y 좌표의 최솟값 $y_{\min}$ 을 각 블록의 x, y 좌표에서 빼면 됩니다.
- 90도 단위 회전에 대한 정보를 제거하기 위해서는, 테트로미노를 0도, 90도, 180도, 270도 회전한 결과를 만들어서 이들 중에서 lexicographical 최솟값 (또는 최댓값)을 고르면 됩니다. 이때 각각 회전한 결과도 $x_{\min} = 0, y_{\min} = 0$ 을 만족해야 합니다. 이는 90도 회전을 할 때 $(x_i, y_i) := (y_{\max} - y_i, x_i)$ 와 같이 구현하면 됩니다.

1M. 한별이의 퍼펙트 수열과 쿼리 교실

data structures, segtree, lazyprop

출제진 의도 – Challenging

- 제출 12번, 정답 3명 (정답률 25.000%)
- 처음 푼 사람: **gs18115**, 48분
- 출제자: index

1M. 한별이의 퍼펙트 수열과 쿼리 교실

- 느리게 갱신되는 세그먼트 트리 자료구조를 이용하면 됩니다.
- 각 노드는 다음과 같은 값을 포함해야 합니다.
 - st 노드가 커버하는 구간의 왼쪽 끝
 - ed 노드가 커버하는 구간의 오른쪽 끝
 - mn 지금까지의 모든 전파를 거친 후, 노드가 커버하는 구간의 원소 중 최솟값
 - mx 지금까지의 모든 전파를 거친 후, 노드가 커버하는 구간의 원소 중 최댓값
 - sum 지금까지의 모든 전파를 거친 후, 노드가 커버하는 구간의 원소들의 합
 - $cnt[i]$ $1 \leq i \leq 10$ 을 만족하는 각 i 에 대하여, 지금까지의 모든 전파를 거친 후, 값 i 를 가지는 구간의 원소들의 개수

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- 또한, 각 노드는 다음과 같은 전파에 대한 정보도 포함해야 합니다.
 - 전파는 항상 (lo, hi, add) 형태로 주어집니다. 만약, 구간의 원소 A_i 에 (lo, hi, add) 라는 전파를 주었을 경우, A_i 는 $\min(\max(A_i, lo), hi) + add$ 로 갱신됩니다.
 - 각 노드는 전파에 대한 정보 lo, hi, add 를 가지고 있어야 하며, 이는 그 노드의 자식 노드들에 전파 (lo, hi, add) 를 나중에 보내줘야 함을 의미합니다.
- 종합하면, 각 노드는 $st, ed, mn, mx, sum, cnt[1, \dots, 10], lo, hi, add$ 라는 값을 포함해야 합니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- 현재 c 번 노드에 $mn_c, mx_c, \dots, lo_c, hi_c, add_c$ 가 저장되어 있고, 부모 노드 p 로부터 전파 (lo_p, hi_p, add_p) 가 전파되었다고 가정합니다.
- (lo_p, hi_p, add_p) 가 전파되고 난 뒤 $mn_c, mx_c, \dots, lo_c, hi_c, add_c$ 가 어떻게 갱신되는지 알아보시다.
- mn_c 와 mx_c 의 갱신은 간단합니다. mn_c 는 $\min(\max(mn_c, lo_p), hi_p) + add_p$ 로, mx_c 는 $\min(\max(mx_c, lo_p), hi_p) + add_p$ 로 갱신됩니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- sum_c 의 갱신은 조금 복잡합니다.
- 우선 sum'_c 을 $sum_c - 1 \times cnt_c[1] - 2 \times cnt_c[2] \cdots - 10 \times cnt_c[10]$ 으로 정의합니다. sum'_c 은 10보다 큰 수들의 합을 의미합니다.
- 또한 cnt'_c 을 $(ed - st + 1) - cnt_c[1] - cnt_c[2] \cdots - cnt_c[10]$ 으로 정의합니다. cnt'_c 은 10보다 큰 수들의 개수를 의미합니다.
- $1 \leq i \leq 10$ 을 만족하는 모든 i 에 대해, $f(i)$ 를 $\min(\max(i, lo_p), hi_p) + add_p$ 로 정의합니다. 이때, Z 를 $f(1) \times cnt_c[1] + f(2) \times cnt_c[2] + \cdots + f(10) \times cnt_c[10]$ 라고 합니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- 주어지는 조건으로 hi_p 는 ∞ 혹은 10 이하의 값만 가짐을 증명할 수 있습니다. 두 경우에 대해서 sum_c 가 어떻게 갱신되는지 살펴봅시다.
 - hi_p 가 ∞ 인 경우, 10 보다 큰 구간의 원소 x 는 $x + add_p$ 로 갱신됩니다. 따라서, 10 보다 큰 구간의 원소들의 합 sum'_c 은 $sum'_c + add_p \times cnt'_c$ 로 갱신이 되기 때문에, sum_c 는 $Z + sum'_c + add_p \times cnt'_c$ 으로 갱신됩니다.
 - hi_p 가 10 이하인 경우, 10 보다 큰 구간의 원소 x 는 $hi_p + add_p$ 로 갱신됩니다. 따라서, 10 보다 큰 구간의 원소들의 합 sum'_c 은 $(hi_p + add_p) \times cnt'_c$ 로 갱신이 되기 때문에, sum_c 는 $Z + (hi_p + add_p) \times cnt'_c$ 으로 갱신됩니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- $cnt_c[1, \dots, 10]$ 를 갱신하는 과정은 다음과 같습니다.
- 우선 임시로 배열 $temp[1, \dots, 10]$ 을 만들고 값을 0으로 초기화해줍니다.
- i 가 lo_p 보다 작은 값이라고 가정합니다. 그러면, i 라는 값은 전파(lo_p, hi_p, add_p)를 거친 후 $lo_p + add_p$ 로 갱신됩니다. 따라서 만약 $lo_p + add_p$ 가 10이하라면, $temp[lo_p + add_p]$ 에 $cnt_c[1] + cnt_c[2] + \dots + cnt_c[lo_p - 1]$ 을 더해줍니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- i 가 hi_p 보다 큰 값이라고 가정합시다. 그러면, i 라는 값은 전파(lo_p, hi_p, add_p)를 거친 후 $hi_p + add_p$ 로 갱신됩니다. 따라서 만약 $hi_p + add_p$ 가 10 이하라면, $temp[hi_p + add_p]$ 에 $cnt_c[hi_p + 1] + cnt_c[hi_p + 2] + \dots + cnt_c[10]$ 을 더해줍니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- i 가 lo_p 이상 hi_p 이하라고 가정합니다. 그러면, i 라는 값은 전파(lo_p, hi_p, add_p)를 거친 후 $i + add_p$ 로 갱신됩니다. 따라서, 각 i 마다 만약 $i + add_p$ 가 10 이하라면, $temp[i + add_p]$ 에 $cnt_c[i]$ 를 더해줍니다.
- 위 과정을 모두 거친 후, $cnt_c[i]$ 를 $temp[i]$ 로 갱신해주면 됩니다.

1M. 한별이의 퍼펙트 수열과 쿼리 교실

- 전파해야 하는 값 (lo_c, hi_c, add_c) 는 어떻게 변할까요?
 - $lo_p > hi_c + add_c$ 인 경우 lo_c 와 hi_c 는 모두 $lo_p - add_c$ 가 됩니다.
 - $hi_p < lo_c + add_c$ 인 경우 lo_c 와 hi_c 는 모두 $hi_p - add_c$ 가 됩니다.
 - 그 외의 경우 lo_c 는 $\max(lo_c, lo_p - add_c)$ 가, hi_c 는 $\min(hi_c, hi_p - add_c)$ 가 됩니다.
 - add_c 는 $add_c + add_p$ 로 갱신됩니다.

1M. 한별이의 퍼펙트 수열과 쿼리 교실

- 이제 주어진 수열과 쿼리 문제를 위에서 정의한 느리게 갱신되는 세그먼트 트리 구조로 바꿔봅시다.
- 우선 초기에 주어진 A_i 값들로 세그먼트 트리를 초기화한 다음 쿼리를 받습니다.
- 1번 쿼리가 들어오면 구간 $[L \dots R]$ 을 커버하는 노드에 대해 전파 $(-\infty, X, 0)$ 를 줍니다.
- 2번 쿼리가 들어오면 구간 $[L \dots R]$ 을 커버하는 노드에 대해 전파 $(X, \infty, 0)$ 를 줍니다.
- 3번 쿼리가 들어오면 구간 $[L \dots R]$ 을 커버하는 노드에 대해 전파 $(-\infty, \infty, X)$ 를 줍니다.
- 4번 쿼리가 들어오면 구간 $[L \dots R]$ 을 커버하는 노드에 대해 lo 값의 최솟값을 출력합니다.
- 5번 쿼리가 들어오면 구간 $[L \dots R]$ 을 커버하는 노드에 대해 hi 값의 최댓값을 출력합니다.
- 6번 쿼리가 들어오면 구간 $[L \dots R]$ 을 커버하는 노드에 대해 sum 값의 합을 출력합니다.

1M. 한별이의 퍼펙트 수열과 퀴리 교실

- 이 문서에 사용된 방식 외에도 여러 다른 구현이 가능합니다.
- 하지만 구현 방식에 따른 속도 차이가 크기 때문에 최적화에 주의합니다.
- 에디토리얼에 주어진 방법 외에도 제곱근 분할이나 Segment Tree Beats 기법을 이용하는 등의 방법이 있습니다.